



Fast Pattern Matching in Strings - KMP Algorithm

A revolutionary algorithm by Knuth, Morris, and Pratt that finds all occurrences of a pattern within text in linear time, without backing up the input.

ANUSKA NATH

002311001003

Information Technology UG-3

The String Matching Problem

Imagine you have a vast document (the **text**) and you need to locate a specific word or phrase (the **pattern**). The fundamental challenge is to efficiently find the **first occurrence** of this pattern within the text.

This seemingly simple task is critical across many fields, from text editors to bioinformatics. While the straightforward naive approach exists, it can be inefficient. The KMP algorithm offers a faster, more optimized solution.



The Naive Approach

The simplest way to find a pattern within a text involves systematically checking every possible alignment. We begin by aligning the pattern with the start of the text.

We then compare characters from left to right. If a mismatch occurs, we simply shift the pattern one position to the right in the text and restart the comparison from the pattern's beginning. This continues until a match is found or all possible alignments are exhausted.

While intuitive, this method can be inefficient. In the worst-case scenario, it can lead to many redundant comparisons, significantly slowing down the search.

Limitation of Naive Approach

Traditional Approach

Try matching at every position, back up when mismatches occur. For pattern $a^n b$ in text $a^{2^n} b$, this makes $(n+1)^2$ character comparisons.

Key Challenge

Backing up input text causes complications with buffering operations. Need a method that scans forward only.

KMP Algorithm - The Core Idea

01

Precompute Shifts

Create a table telling how far to slide the pattern when a mismatch occurs at each position.

02

Slide Smartly

Use the table to skip ahead efficiently when mismatches happen, avoiding unnecessary comparisons.

03

Scan Forward

Move through text once without backing up, making only $O(m+n)$ comparisons total.

The Prefix-Suffix Concept

The genius of KMP lies in analyzing the pattern's internal structure. For any given prefix of our search pattern, we find the **longest proper prefix** that is also a **suffix** of that same prefix. This pre-computed knowledge allows the algorithm to determine the optimal shift amount after a mismatch, preventing redundant comparisons and ensuring a linear time complexity.



Why KMP Works

Reuse Previous Matches

- When a mismatch occurs, we don't throw away the matches we already found.
- The LPS table tells us how much of the pattern can still be reused.

Prefix = Suffix Knowledge

- The pattern's internal structure stores the longest proper prefix that is also a suffix.
- KMP knows which earlier characters matter without checking them again.

No Backtracking in Text

- Unlike the naive approach, the text pointer never moves backward.
- Only the pattern pointer shifts back through the LPS table, which is much cheaper.

Pattern Position Encodes History

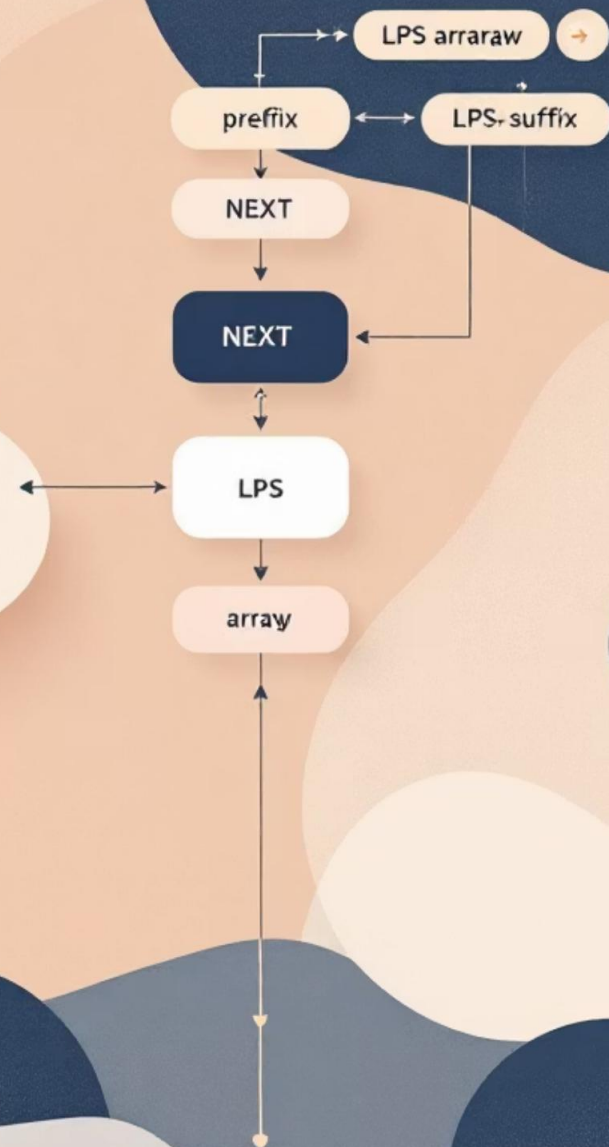
- The current pattern position encodes what has matched so far.
- We do not need to store or re-examine that history separately.

The NEXT Table (LPS Array)

The KMP algorithm constructs a "NEXT" table (also known as the Longest Proper Prefix (LPS) array) for the pattern. For each position i in the pattern, $LPS[i]$ stores the length of the longest proper prefix of $pattern[0 \dots i]$ that is also a suffix of $pattern[0 \dots i]$.

This table is pre-computed based solely on the pattern itself, before any comparisons with the text begin. It essentially encodes the internal symmetries and redundancies within the pattern.

When a mismatch occurs at a certain position in the text, instead of simply shifting the pattern by one and restarting the comparison, the KMP algorithm uses the LPS value at the point of mismatch to intelligently shift the pattern. This ensures that the pattern is moved to the furthest possible position while still maintaining a valid partial match, thereby avoiding unnecessary re-comparisons of characters that are already known to match.



Building the NEXT Table: An Example

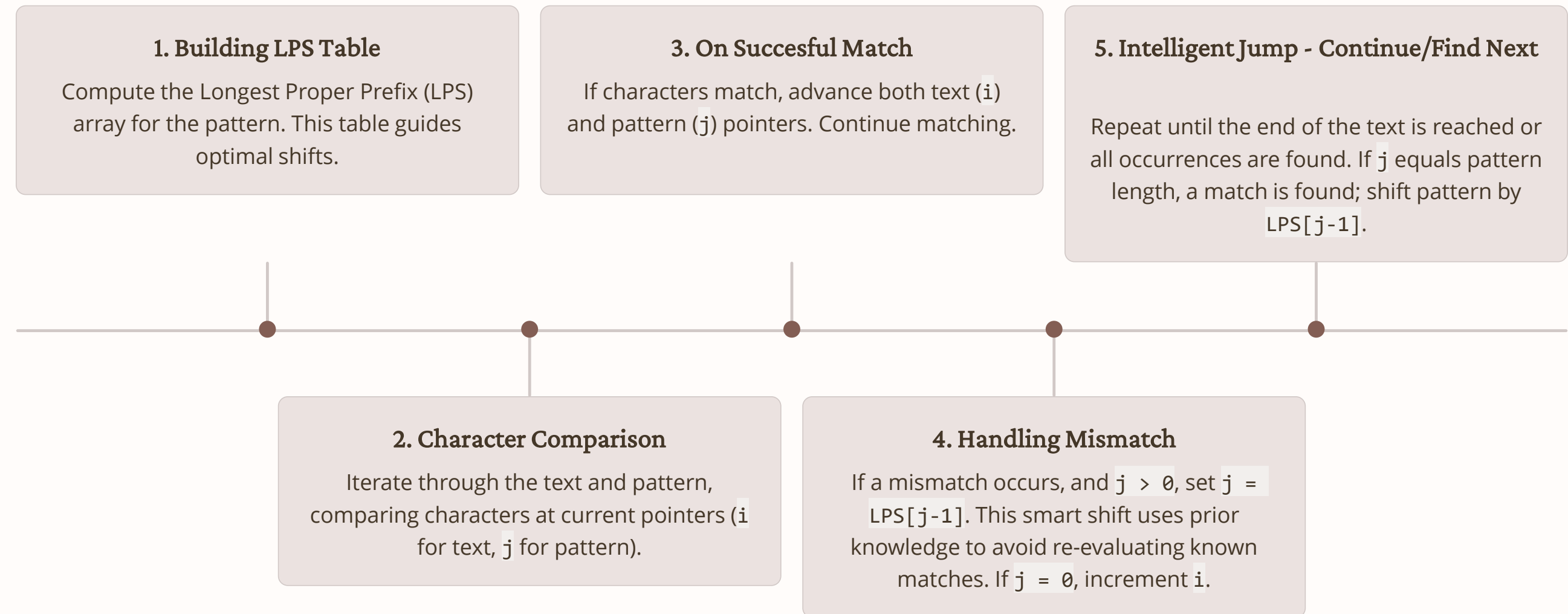
Let's walk through the computation of the LPS (NEXT) table for the pattern "ababd". This table guides KMP on how far to shift the pattern after a mismatch.

01	02	03
Prefix "a"	Prefix "ab"	Prefix "aba"
LPS[0] = 0 (No proper prefix)	LPS[1] = 0 (Proper prefix "a" does not match suffix "b")	LPS[2] = 1 (Longest proper prefix "a" matches suffix "a")
04	05	
Prefix "abab"	Prefix "ababd"	
LPS[3] = 2 (Longest proper prefix "ab" matches suffix "ab")	LPS[4] = 0 (No proper prefix matches a suffix)	

This computed LPS table for "ababd" would be [0, 0, 1, 2, 0]. Each value represents the length of the longest proper prefix of the sub-pattern that is also a suffix of that sub-pattern.

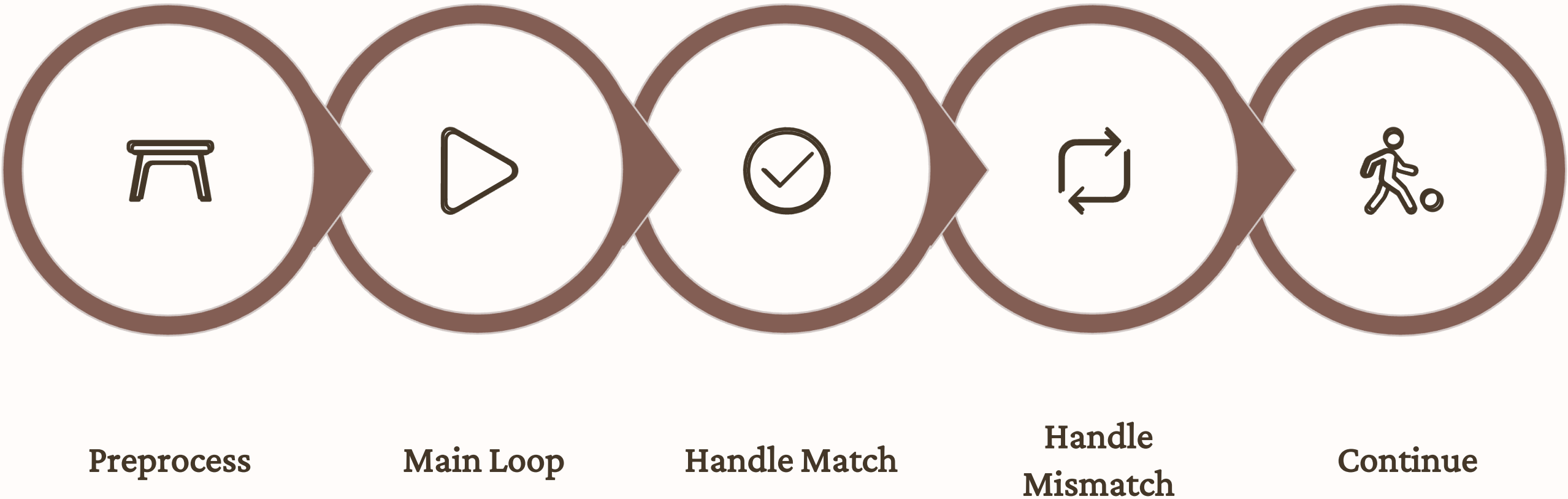
KMP: How It Works

Once the NEXT (LPS) table is built, the KMP algorithm proceeds with matching the pattern against the text. Its efficiency comes from intelligently utilizing this pre-computed table when mismatches occur, preventing unnecessary comparisons.



KMP Algorithm: Step-by-Step Flow

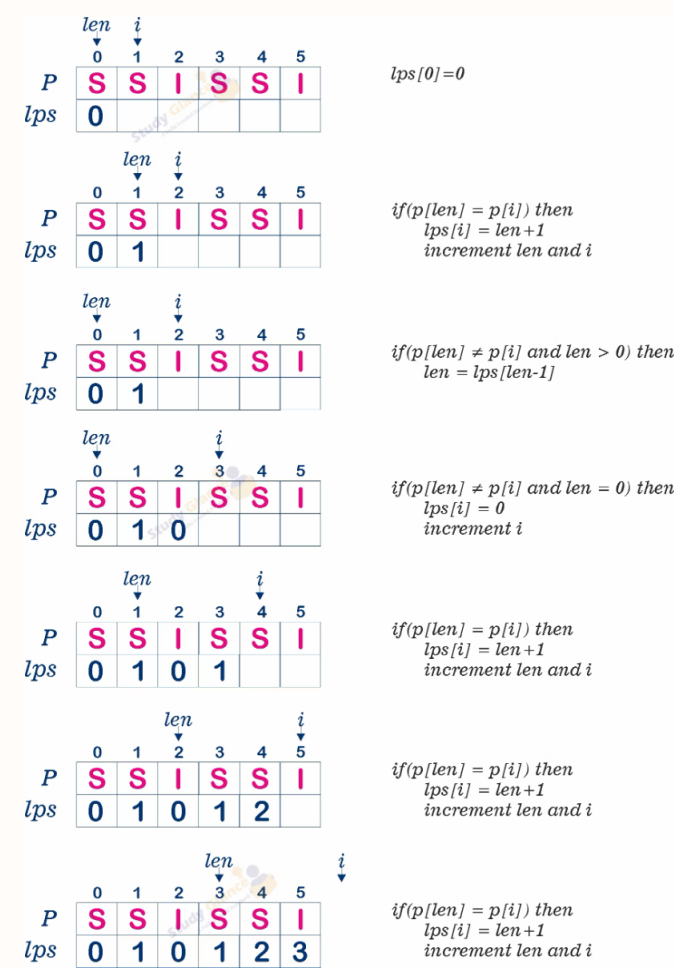
Understanding the KMP algorithm involves following a precise sequence of operations. This flow outlines how the precomputed NEXT (LPS) table is utilized to achieve efficient string matching.



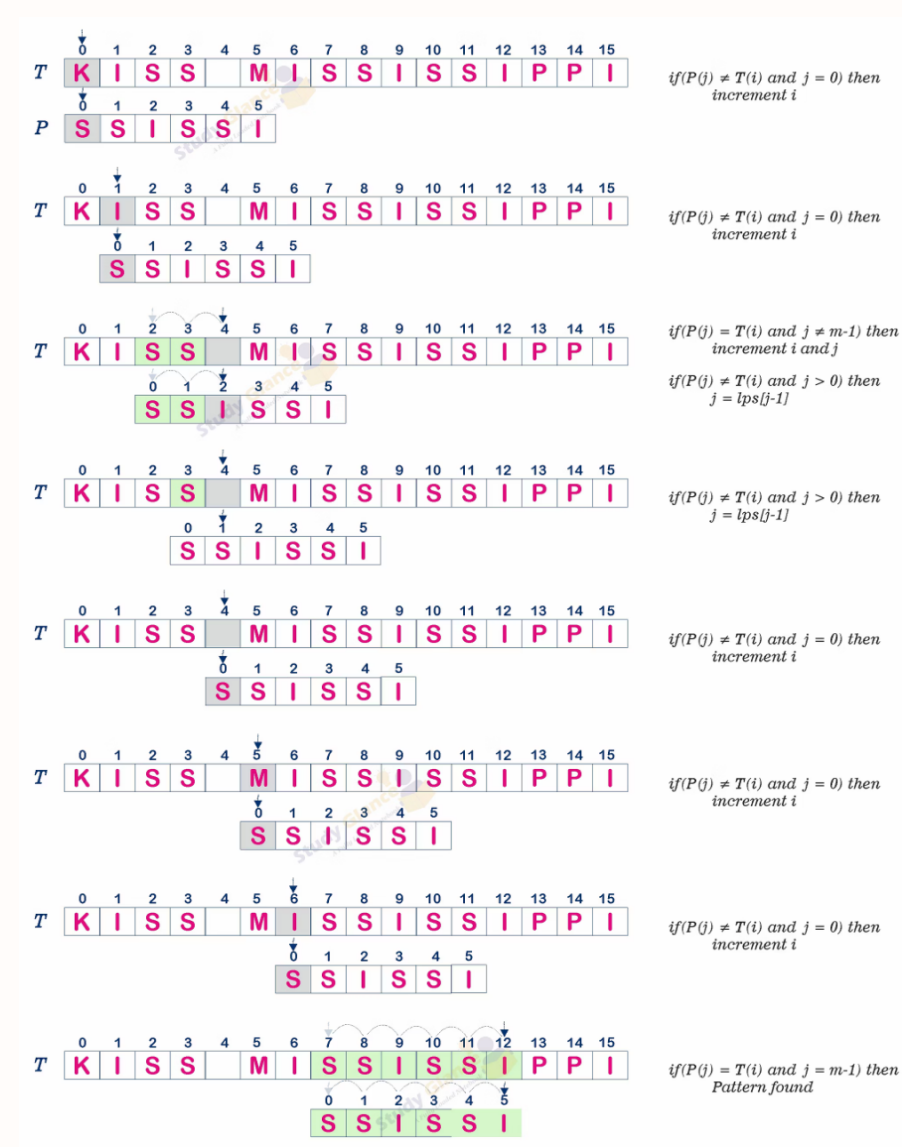
KMP Algorithm: Step-by-Step Example

String: KISS MISSISSIPPI, Pattern: SSISSI

LPS Array Construction:



KMP Algorithm step by step:



Algorithm Performance

$$O(m+n)$$

Time Complexity

Proportional to sum of pattern and text lengths

- **No backtracking:** Unlike the naive approach, we never re-examine characters in the text.
- **Each character processed once:** The text pointer only moves forward.
- **Pattern preprocessing:** $O(m)$ to build the LPS table.
- **Main matching:** $O(n)$ to scan through the text.
- **Total:** $O(m) + O(n) = O(m+n)$.

$$O(m)$$

Memory

Internal memory needed for pattern matching

- **LPS array stores one value per pattern character:** Single integer per character, with no growth as the text gets longer.
- **No additional data structures are needed:** Only pointers and counters are used beyond the LPS array.
- **Independent of text length:** Memory usage depends only on pattern length m , not text length n .

Constants independent of alphabet size. At most $2n$ steps after table setup.

Naive v/s KMP: A Comparison

The core differences between the naive string matching approach and the optimized KMP algorithm are summarized below, highlighting KMP's significant performance advantages.

Feature	Naive	KMP
Time Complexity	$O(mn)$	$O(m+n)$
Backtracking	Yes	No
Efficiency	Poor	Optimal

KMP's pre-computation of the LPS array eliminates redundant comparisons, making it far more efficient for large texts and patterns with repeating elements.

Extensions & Variations



Find All Matches

Modify to find every occurrence, not just leftmost match.



Multiple Patterns

Use trie memory to search for several patterns simultaneously in $O(n)$ time.



Word-Level Matching

Treat entire words as "supercharacters" for packed text architectures.

Practical Benefits



Linear Time

Runs in $O(m+n)$ time regardless of pattern structure



No Backup

Scans forward only, ideal for streaming data and buffering



Efficient Memory

Uses only $O(m)$ internal storage for pattern

Particularly valuable in text-editing programs that frequently search large documents.

Key Takeaways

1

No Backing Up Text

The text pointer never moves backward, only forward.

2

Preprocessing Pattern

Build the LPS table once before matching begins.

3

Linear time guaranteed

$O(m+n)$ complexity regardless of input patterns.

4

Works Independent of Alphabet size

Algorithm performance doesn't depend on character set size.

